

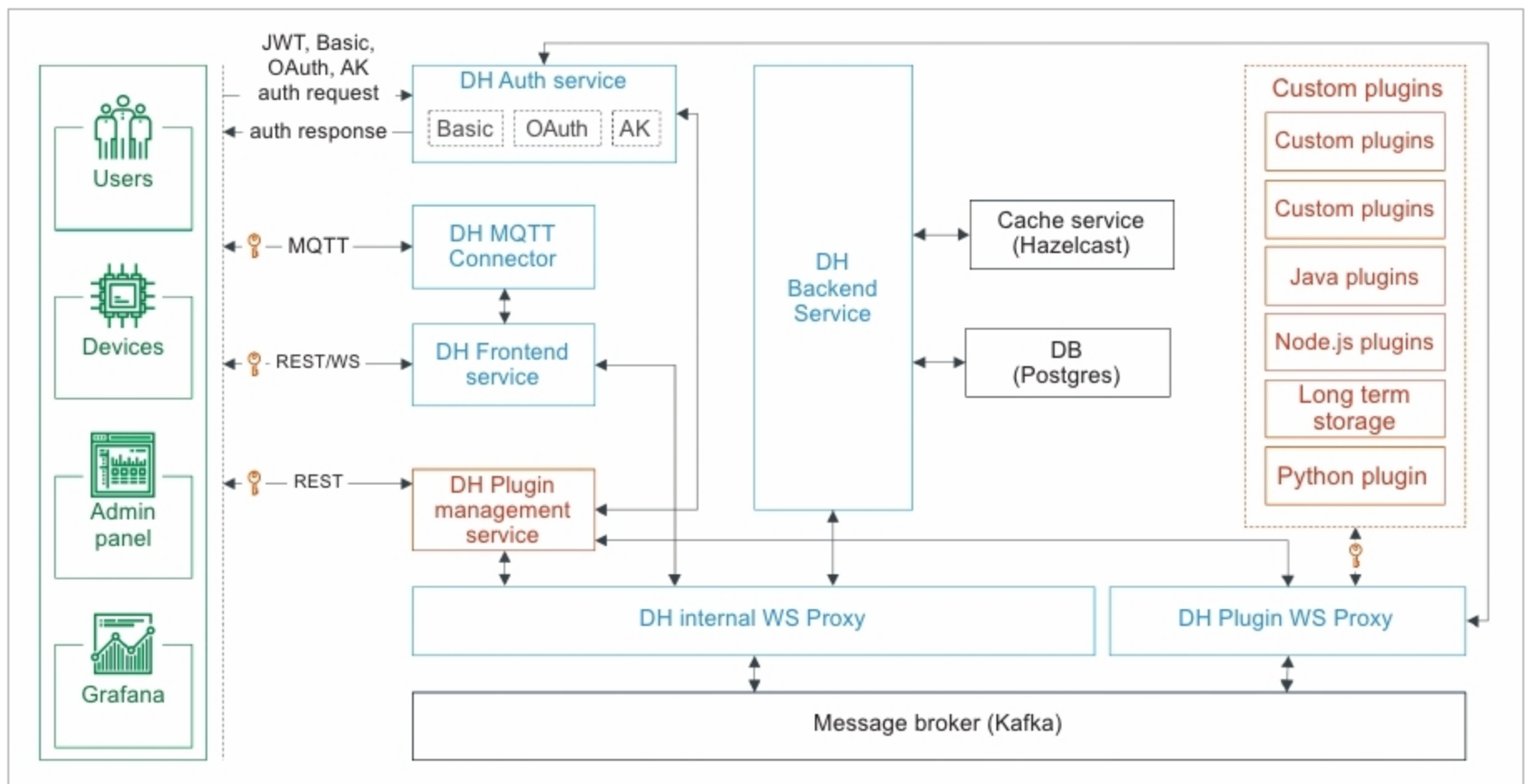


Figure 1: DeviceHive microservices architecture

- **PostgreSQL:** This is the backend database to store all the data with regard to devices, networks, users, device types and configuration settings.
- **Hazelcast IMDG:** This is a clustered, in-memory data grid that uses sharding for data distribution and supports monitoring. All notifications are saved to a distributed cache storage for speedy access, and this can be removed in 2 minutes.
- **Message Bus (Kafka):** This supports communication between services and load balancing, as Kafka is a fast, distributed and fault-tolerant messaging system. In DeviceHive, the WebSocket Kafka proxy is used. It is written in Node.js because of flexibility in messaging.
- **DeviceHive frontend service:** This supports the RESTful and WebSocket APIs, performing all sorts of primary checks, sending requests to backend services and delivering responses in an asynchronous manner.
- **DeviceHive backend service:** This stores data in Hazelcast to manage subscriptions and retrieve data via requests from other services, from Hazelcast or from the database.
- **DeviceHive Auth service:** This contains information regarding the access control of users, the devices connected, network types and device types. It provides the RESTful API for generating, validating and refreshing tokens.
- **DeviceHive plugin service:** DeviceHive plugin support can help users to register devices, and define network types with the required JWT tokens. All the plugins are created in Node.js, Python and Java.

DeviceHive API

DeviceHive API acts as a central component of the framework to facilitate communication and interaction with varied components. The API is responsible for providing access to information regarding all the components registered in the system in order to exchange messages in real-time scenarios.

The DeviceHive API has three types of consumers:

- Client
- Administrator
- Device

Client: This is regarded as an application to control and administer devices. It can be an interface or software to manage the entire network.

Administrator: This controls the whole environment with full access to all components. It can create and manage API users, device networks, and all notifications and commands.

Device: This is termed as an individual unit with a unique identifier, name and other meta-information to communicate with the API. It takes commands from other components and executes them in an efficient manner. **END** 🐧

References

- [1] <http://devicehive.com>
- [2] <https://github.com/devicehive>

By: Anand Nayyar

The author works at Duy Tan University in Vietnam. He loves to explore open source technologies, sensor communications, network security and the Internet of Things.